ShopEZ

One-Stop E-Commerce Platform

Full Stack Development with MERN — Project Documentation

1. Introduction

Project Title: ShopEZ – One Stop E-Commerce Platform

Team Members:

- Himabindhu Ravuri — [Front end Developer]
- P Humera – [Backend Developer]
- Munagala Jyothi – [Database]
- Paidimuddala Jyothish – [Testing and Integration]
- Kolla Hemanth Reddy – [Documentation]

2. Project Overview

Purpose

ShopEZ is a full-stack MERN e-commerce application that allows customers to browse products, add them to a cart, place orders, and complete checkout. The goal of the project is to provide a complete, realistic online shopping experience — from product discovery to order tracking — while also giving an administrator the tools to manage the product catalogue through a dedicated admin panel.

Features

- Product catalogue with categories, search, filters, and stock status
- Shopping cart with quantity controls
- Checkout flow with address entry and order summary
- Order history with delivery tracking
- Login/Register secured with Firebase Authentication
- Admin panel for product CRUD, protected by email-based access control
- Responsive design across devices

3. Architecture

Frontend

The frontend is built with React.js using a component-based architecture. Navigation between pages is handled by React Router DOM, and application-wide cart state (adding, removing, and updating item quantities) is managed through React's Context API via a dedicated CartContext. Axios is used to communicate with the backend REST API, and Firebase Authentication handles user login and registration on the client side.

Backend

The backend is a Node.js application built with Express.js, exposing a REST API that the frontend consumes. It is organized into models (Mongoose schemas) and routes (Express route handlers) with a central server.js entry point that wires up middleware, database connection, and API routes.

Database

MongoDB is used as the primary data store, accessed through Mongoose object modelling. The core schemas are Product and Order:

- Product — represents catalogue items (name, category, price, stock, image, etc.)
- Order — represents a customer's placed order, including items, address, and delivery/order status

4. Setup Instructions

Prerequisites

- Node.js (v16 or later recommended)
- npm (comes with Node.js)
- MongoDB (local instance or MongoDB Atlas cluster)
- A Firebase project (for Authentication)

Installation

Backend:

```
cd shopez-backend
npm install
# Create a .env file with:
# MONGO_URI=your_mongodb_uri
# PORT=5000
node server.js
```

Frontend:

```
cd shopez
npm install
# Create a .env file with your Firebase config (see below)
npm start
```

Environment Variables

Frontend (.env):

```
REACT_APP_FIREBASE_API_KEY=
REACT_APP_FIREBASE_AUTH_DOMAIN=
REACT_APP_FIREBASE_PROJECT_ID=
REACT_APP_FIREBASE_STORAGE_BUCKET=
REACT_APP_FIREBASE_MESSAGING_SENDER_ID=
REACT_APP_FIREBASE_APP_ID=
```

Backend (.env):

```
MONGO_URI=
PORT=5000
```

5. Folder Structure

Client (shopez/)

```
shopez/
└── src/
    ├── components/   # Navbar, Footer
    ├── context/      # CartContext (global cart state)
    ├── pages/        # Home, Cart, Checkout, Orders, Admin, Login
    └── firebase.js   # Firebase configuration
```

Server (shopez-backend/)

```
shopez-backend/
├── models/    # Product, Order schemas (Mongoose)
├── routes/    # products, orders API routes
└── server.js  # Application entry point
```

6. Running the Application

Server	Directory	Command
Frontend	shopez/	npm start
Backend	shopez-backend/	node server.js

By default the backend runs on the port defined in its .env file (e.g. 5000) and the React frontend runs on http://localhost:3000, proxying API requests to the backend.

7. API Documentation

The backend exposes REST endpoints for products and orders. Fill in the table below with the finalized routes from shopez-backend/routes/.

8. Authentication

User authentication and authorization are handled on the client side using Firebase Authentication. Users can register and log in via Firebase, which issues an identity token that identifies the current session.

- Login/Register — implemented via Firebase Auth SDK on the Login page
- Session handling — Firebase manages the authenticated session/token on the client
- Admin access control — the admin panel is protected by checking the logged-in user's email against an authorized admin email, rather than a separate role field

9. User Interface

Key screens include the Home page (product catalog), Cart page, Checkout page, Order History page, Admin panel, and Login page. Screenshots for each are available in the repository's screenshots/ folder and can be inserted below:

- Home Page — product browsing, categories, search, and filters
- Cart Page — items with quantity controls
- Checkout Page — address entry and order summary
- Order History — past orders with delivery tracking

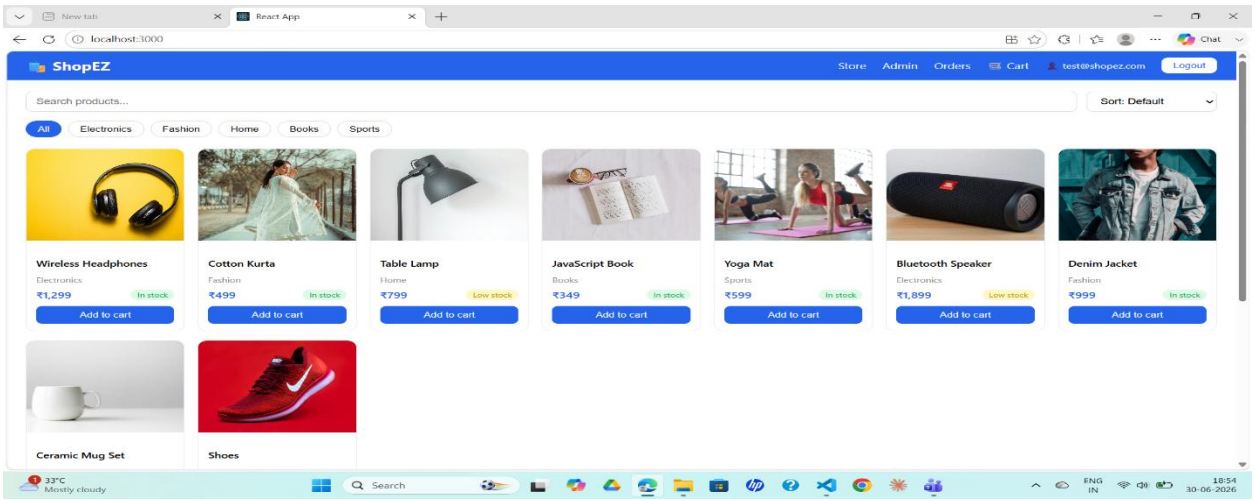
- Admin Panel — product CRUD interface
- Login Page — Firebase-based login/register

10. Testing

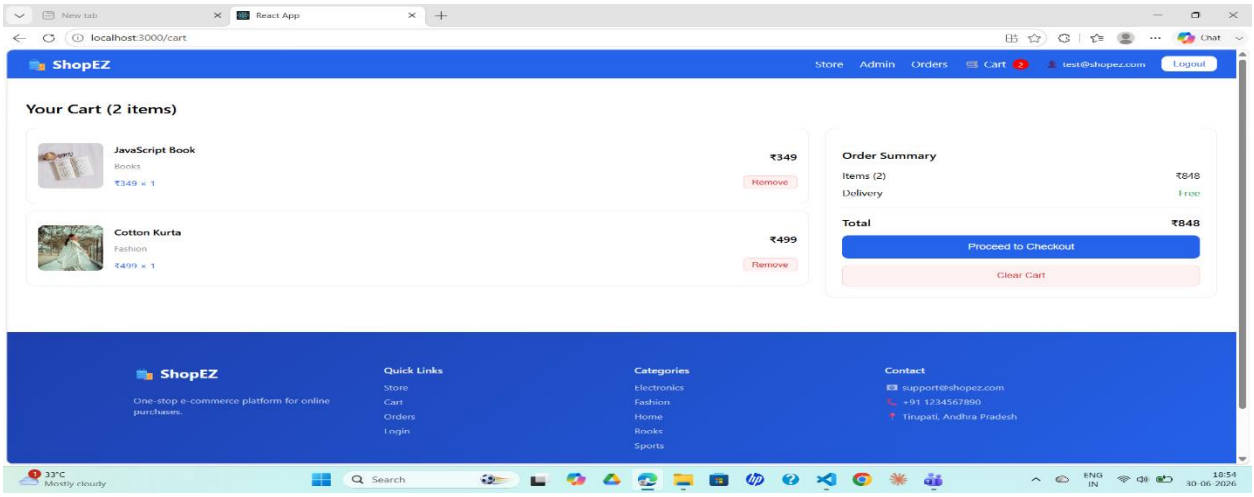
Manual testing was carried out across core flows: product browsing/search/filters, cart (add/update/remove), checkout and order placement, order history, Firebase login/register, and admin CRUD access. Backend API endpoints were tested independently using Postman for expected responses and error handling. No automated test suite (Jest/Supertest) or CI pipeline is set up yet.

11. Screenshots or Demo

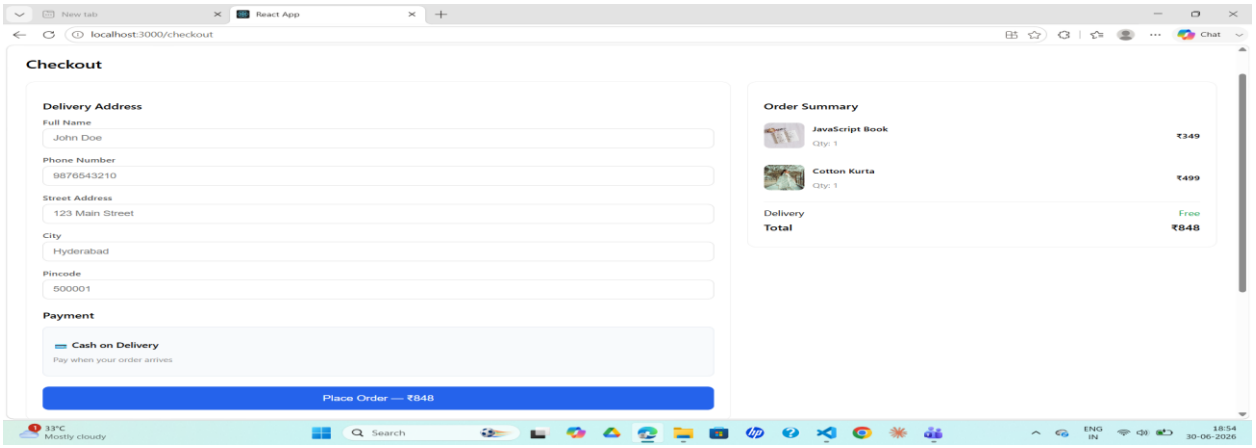
Home Page



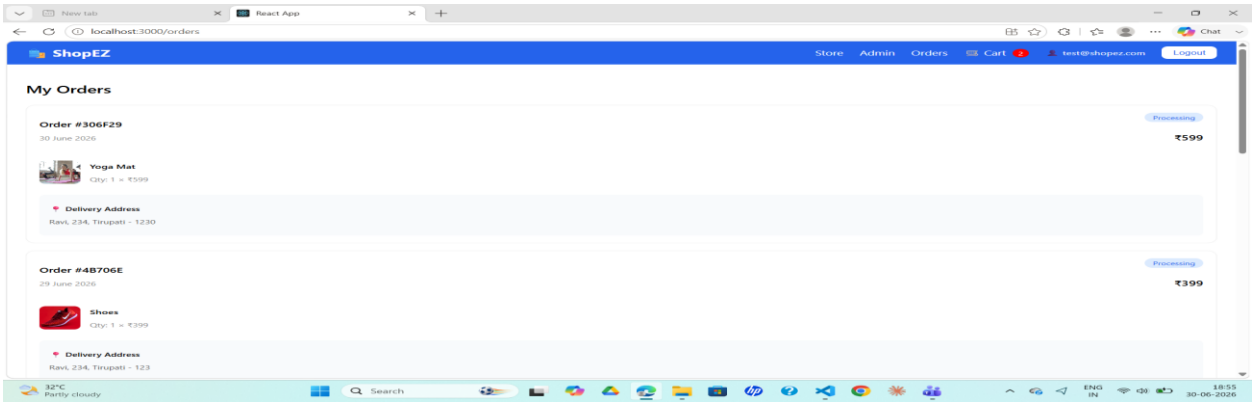
Cart Page



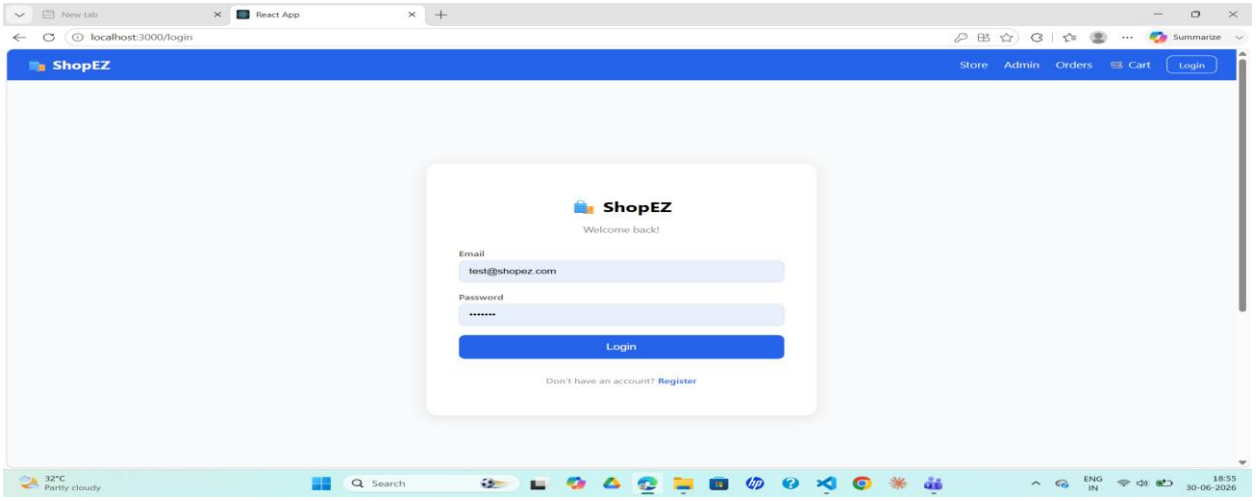
Checkout Page



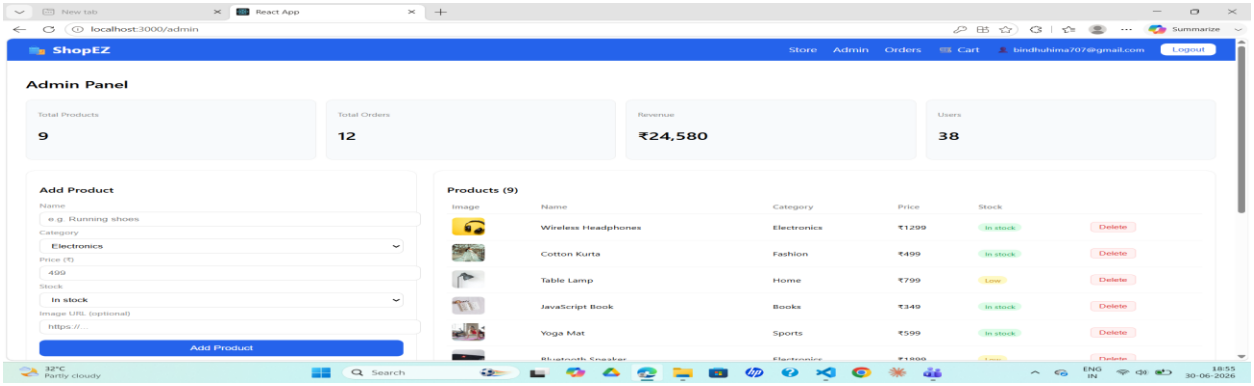
Order Page



Login Page



Admin Page



12. Known Issues

- Only Cash on Delivery is supported; no real payment gateway integrated
- Cart is not persisted server-side, so it resets on refresh/device switch
- Order status must be updated manually by admin, no auto notifications
- No email/SMS notifications for order confirmation or status changes
- Admin access is based on a whitelisted email, not proper role-based auth

13. Future Enhancements

- Integrate a real payment gateway (Razorpay/Stripe) instead of Cash on Delivery
- Add product reviews and ratings
- Add a Wishlist feature
- Advanced filters (price range, ratings)
- Admin dashboard with sales charts and analytics
- Email notifications for order confirmation
- Order status tracking (Processing → Shipped → Delivered) updated by admin
- Dark mode support
- Multi-language support